Enrolment No./Seat No_____

# GUJARAT TECHNOLOGICAL UNIVERSITY

**BE - SEMESTER–V (NEW) EXAMINATION – SUMMER 2024**

**Subject Code:3150703 Date:31-05-2024**

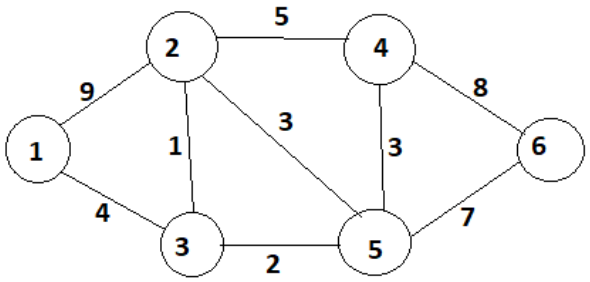
**Subject Name:Analysis and Design of Algorithms**

**Time:02:30 PM TO 05:00 PM Total Marks:70**

**Instructions:**

1. **Attempt all questions.**
2. **Make suitable assumptions wherever necessary.**
3. **Figures to the right indicate full marks.**
4. **Simple and non-programmable scientific calculators are allowed.**

| | | | MARKS |
|---|---|---|---|
| **Q.1** | **(a)** | Find Omega (Ω) notation of function $f(n)=2n^2 + n * \lg n + 6n$ | **03** |
| | **(b)** | Define Big-oh and Theta notations with graph | **04** |
| | **(c)** | Write and analyze an insertion sort algorithm to sort n items into ascending order. | **07** |
| **Q.2** | **(a)** | Explain: Articulation Point, Graph, Tree | **03** |
| | **(b)** | Solve following recurrence using Master Theorem: $T(n) = 3T(n/3) + n^3$. | **04** |
| | **(c)** | Illustrate the working of the quick sort on input instance: 25, 29, 30, 35, 42, 47, 50, 52, 60. Comment on the nature of input i.e. best case, average case or worst case. | **07** |
| | | **OR** | |
| | **(c)** | What is recurrence? Explain recursion-tree method with suitable example. | **07** |
| **Q.3** | **(a)** | What is Principle of Optimality? Explain its use in Dynamic Programming Method | **03** |
| | **(b)** | Explain Binomial Coefficient algorithm using dynamic programming. | **04** |
| | **(c)** | Obtain longest common subsequence using dynamic programming. Given A = “acabaca” and B = “bacac”. | **07** |
| | | **OR** | |
| **Q.3** | **(a)** | Explain the steps of greedy strategy for solving a problem | **03** |
| | **(b)** | Demonstrate Binary Search method to search Key = 14, form the array A=<2,4,7,8,9,10,12,14,18> | **04** |
| | **(c)** | What is a minimum spanning tree? Draw the minimum spanning tree correspond to following graph using Prim’s algorithm. | **07** |

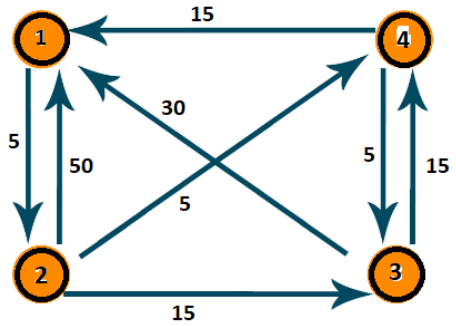



| | | | |
|---|---|---|---|
| **Q.4** | **(a)** | Explain Tower of Hanoi Problem, Derive its recursion equation and computer it’s time complexity. | **03** |
| | **(b)** | Using greedy algorithm find an optimal schedule for following jobs with n=7 profits: (P1, P2, P3, P4, P5, P6, P7) = (3, 5, 18, 20, 6, 1, 38) and deadline (d1, d2, d3, d4, d5, d6, d7) = (1, 3, 3, 4, 1, 2, 1) | **04** |

| | | | |
|---|---|---|---|
| | **(c)** | Solve the following Knapsack Problem using greedy method. Number of items = 5, knapsack capacity W = 100, weight vector = {50, 40, 30, 20, 10} and profit vector = {1, 2, 3, 4, 5}. | **07** |

**OR**

| | | | |
|---|---|---|---|
| **Q.4** | **(a)** | What are the disadvantages of greedy method over dynamic programming method? | **03** |
| | **(b)** | Find an optimal Huffman code for the following set of frequency. A : 50, b: 20, c: 15, d: 30 | **04** |
| | **(c)** | Consider a Knapsack with the maximum weight capacity M is 7, for the objects with value [12,10,20,15] with weights [2,1,3,2] solve using dynamic programming the maximum value the knapsack can have. | **07** |
| **Q.5** | **(a)** | Define NP-Complete and NP-Hard problems. | **03** |
| | **(b)** | Draw the state space tree Diagram for 4 Queens problem. | **04** |
| | **(c)** | Find all pair of shortest path using Floyd's Algorithm for following graph. | **07** |




**OR**

| | | | |
|---|---|---|---|
| **Q.5** | **(a)** | Define BFS. How it is differ from DFS. | **03** |
| | **(b)** | Explain the naive string matching algorithm. | **04** |
| | **(c)** | Explain spurious hits in Rabin-Karp string matching algorithm with example. Working modulo q=13, how many spurious hits does the Rabin Karp matcher encounter in the text T = 2359023141526739921 when looking for the pattern P = 31415? | **07** |

**********

Q.1 (a) Find Omega (Ω) notation of function f(n)=2n2 + n * lg n + 6n

**Ω (Omega) Notation of the Given Function**

**Given function:**

$$f(n) = 2n^2 + n \cdot \lg\, n + 6n$$

**Definition of Ω (Omega) Notation:**
Ω-notation is used to represent the **asymptotic lower bound** of a function. It describes the **minimum growth rate** of an algorithm and shows the **best-case or guaranteed performance** for sufficiently large values of input size $n$.

**Step-by-step Explanation:**

- The given function contains three terms:
  - $2n^2$(quadratic term)
  - $n\lg\, n$(log-linear term)
  - $6n$(linear term)
- As $n$becomes very large, the term with the **highest growth rate** dominates the function.
- Among all terms, $2n^2$grows faster than $n\lg\, n$and $6n$.
- Therefore, for sufficiently large $n$,

$$f(n) \geq 2n^2$$

- Let $c = 2$and $n_0 = 1$, then

$$f(n) \geq c \cdot n^2 \text{for all } n \geq n_0$$

(b) Define Big-oh and Theta notations with graph (4 mark)

**1) Big-Oh (O) Notation**

**Definition:**
Big-Oh notation is used to describe the **upper bound** of an algorithm's time or space complexity. It indicates the **worst-case growth rate** of a function as the input size $n$becomes very large.

**Explanation:**

- Big-Oh gives an upper limit on how fast a function can grow.
- It helps in analyzing the **maximum time or space** an algorithm may require.
- Formally, a function $f(n)$is said to be $O(g(n))$if there exist positive constants $c$and $n_0$such that:

$$f(n) \leq c \cdot g(n) \text{for all } n \geq n_0$$

- It ignores constants and lower-order terms.

**Uses / Importance:**

- Used to compare algorithms.
- Ensures performance will not exceed a certain limit.
- Commonly used in **worst-case analysis**.

**Graph Description:**

- On the graph, the function $f(n)$lies **below or equal to** the curve $c \cdot g(n)$after some value of $n_0$.
- The curve $c \cdot g(n)$acts as an **upper boundary**.

---

**2) Theta (Θ) Notation**

**Definition:**
Theta notation represents the **tight bound** of an algorithm. It specifies

that a function grows at the **same rate** as another function, providing both **upper and lower bounds**.

**Explanation:**

- Theta gives an **exact asymptotic behavior** of a function.
- A function $f(n)$is said to be $\Theta(g(n))$if there exist positive constants $c_1, c_2$, and $n_0$such that:
- 
- It means $f(n)$is neither too large nor too small compared to $g(n)$.

**Uses / Importance:**

- Provides precise complexity analysis.
- Used when best-case and worst-case complexities are the same.
- Helpful for accurate algorithm comparison.

**Graph Description:**

- On the graph, the function $f(n)$lies **between two curves** $c_1 g(n)$and $c_2 g(n)$after $n_0$.
- These two curves form a **tight band** around $f(n)$.

(c) Write and analyze an insertion sort algorithm to sort n items into ascending order. (7 mark)

**Insertion Sort Algorithm**

**Introduction / Definition:**

Insertion sort is a **simple comparison-based sorting algorithm** that sorts elements by **inserting each element into its correct position** in a partially sorted list. It works in a way similar to sorting playing cards in hand, where each new card is placed at its appropriate position among the already sorted cards.

---

**Detailed Explanation:**

- In insertion sort, the array is divided into two parts:
  - **Sorted sublist** (initially containing the first element)
  - **Unsorted sublist** (remaining elements)
- The algorithm repeatedly takes one element from the unsorted sublist and inserts it into the correct position in the sorted sublist.
- This process continues until all elements are sorted in ascending order.
- It is an **in-place** and **stable** sorting algorithm.

---

**Algorithm (Steps in Words):**

1. Start from the **second element** of the array, assuming the first element is already sorted.
2. Store the current element as the **key**.
3. Compare the key with elements in the sorted part (to its left).
4. Shift all elements that are **greater than the key** one position to the right.
5. Insert the key at its **correct position**.
6. Repeat the above steps for all remaining elements until the array is fully sorted.

---

**Working Explanation (Conceptual):**

- The algorithm checks each element and places it in the proper position among previously sorted elements.

- If the element is smaller, shifting occurs; if larger, it remains in place.
- After each pass, the size of the sorted sublist increases by one.
- A diagram would show elements moving right while the key is inserted into the correct position.

---

**Example (Explained in Words):**

Consider an array: **8, 3, 5, 2**

- First pass: 8 is considered sorted.
- Second pass: 3 is compared with 8 → inserted before it → 3, 8
- Third pass: 5 is placed between 3 and 8 → 3, 5, 8
- Fourth pass: 2 is placed at the beginning → 2, 3, 5, 8
  Thus, the array is sorted in ascending order.

---

**Analysis of Insertion Sort:**

**Time Complexity:**

- **Best Case:** $O(n)$
  - Occurs when the array is already sorted.
- **Average Case:** $O(n^2)$
  - Occurs for randomly ordered elements.
- **Worst Case:** $O(n^2)$
  - Occurs when the array is sorted in reverse order.

**Space Complexity:**

- $O(1)$, as it requires only a constant amount of extra space.

---

**Key Features:**

- Simple and easy to implement.
- Stable sorting algorithm.
- Works efficiently for **small datasets**.
- Suitable for nearly sorted arrays.

---

**Advantages:**

- Easy to understand and implement.
- Efficient for small input sizes.
- Requires no additional memory.

---

**Disadvantages:**

- Inefficient for large datasets.
- Slower compared to advanced sorting algorithms like quick sort and merge sort.

---

**Applications:**

- Used in **small-scale applications**.
- Suitable for **online sorting**, where elements are received one by one.
- Used as a subroutine in hybrid sorting algorithms.

Q.2 (a) Explain: Articulation Point, Graph, Tree (3 mark)

**Articulation Point:**

An articulation point is a **vertex in a graph** whose removal, along with its associated edges, **increases the number of connected components** of the graph. In other words, if removing a vertex disconnects the graph, that vertex is called an articulation point. Articulation points are important in network analysis because they help identify **critical nodes** whose failure can break communication. They are commonly used in computer networks, transportation systems, and circuit design to find vulnerable points. Detecting articulation points helps in improving **reliability and fault tolerance** of networks.

**Graph:**

A graph is a **non-linear data structure** consisting of a set of **vertices (nodes)** and a set of **edges** that connect pairs of vertices. Graphs can be **directed or undirected**, **weighted or unweighted**, depending on the type of relationships they represent. Graphs are used to model real-world problems such as social networks, road maps, communication networks, and scheduling systems. Each edge represents a connection between two vertices, and graphs provide an efficient way to represent complex relationships.

**Tree:**

A tree is a **special type of graph** that is **connected and acyclic**, meaning it has no cycles. In a tree, there is **exactly one unique path** between any two vertices. Trees have a hierarchical structure with a **root node** and child nodes, which makes them useful for representing organizational structures, file systems, and decision processes. A tree with $n$nodes always has $n - 1$edges, and it is widely used in searching and sorting algorithms.

(b) Solve following recurrence using Master Theorem: T(n) = 3T(n/3) + n^3. (4 mark)

**Solving the Recurrence using Master Theorem**

**Given Recurrence Relation:**

$$T(n) = 3T\left(\frac{n}{3}\right) + n^3$$

---

**Step 1: Identify parameters of Master Theorem**

The standard form of Master Theorem is:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Comparing with the given recurrence:

- $a = 3$
- $b = 3$
- $f(n) = n^3$

---

**Step 2: Calculate $n^{\log_b a}$**

$$\log_b a = \log_3 3 = 1$$
$$n^{\log_b a} = n^1 = n$$

---

**Step 3: Compare $f(n)$ with $n^{\log_b a}$**

- $f(n) = n^3$
- $n^3$ grows **faster** than $n$

Thus,

$$f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$$

where $\epsilon = 2$.

---

**Step 4: Apply Master Theorem (Case 3)**

According to **Case 3** of the Master Theorem:

- If $f(n) = \Omega\left(n^{\log_b a+\epsilon}\right)$
- And the **regularity condition** is satisfied,

Then:

$$T(n) = \Theta(f(n))$$

Since $f(n) = n^3$, the regularity condition holds.

---

**Final Answer / Conclusion:**

$$\boxed{T(n) = \Theta(n^3)}$$

(c) Illustrate the working of the quick sort on input instance: 25, 29, 30, 35, 42, 47, 50, 52, 60. Comment on the nature of input i.e. best case, average case or worst case. (7 mark)

**Introduction / Definition:**

Quick sort is a **divide-and-conquer based sorting algorithm** that sorts elements by selecting a **pivot element** and partitioning the array such that elements smaller than the pivot are placed on its left and elements greater than the pivot are placed on its right. The same process is then applied recursively to the sub-arrays.

---

**Principle of Quick Sort:**

- Choose a pivot element from the list (commonly the **first or last element**).

- Rearrange elements so that:
    - All elements less than pivot appear before it.
    - All elements greater than pivot appear after it.
- Place the pivot at its correct position.
- Recursively apply the same steps to left and right sub-arrays.

A diagram would show repeated partitioning of the array around different pivot elements.

---

**Given Input Instance:**

25, 29, 30, 35, 42, 47, 50, 52, 60

The list is already sorted in **ascending order**.

---

**Working of Quick Sort (Step-by-Step Explanation):**

Assume **first element is chosen as pivot**.

**Step 1:**

- Pivot = 25
- All remaining elements are **greater than 25**.
- Pivot 25 stays at the first position.
- Left sub-array = Empty
- Right sub-array = 29, 30, 35, 42, 47, 50, 52, 60

**Step 2:**

- Pivot = 29
- All remaining elements are greater than 29.
- Pivot 29 is placed at its correct position.

- Left sub-array = Empty
- Right sub-array = 30, 35, 42, 47, 50, 52, 60

**Step 3:**

- Pivot = 30
- Left sub-array = Empty
- Right sub-array = 35, 42, 47, 50, 52, 60

**Step 4:**

- Pivot = 35
- Left sub-array = Empty
- Right sub-array = 42, 47, 50, 52, 60

**Further Steps:**

- The same process continues with pivots 42, 47, 50, and 52.
- Each time, the left sub-array remains empty and only the right sub-array is processed.
- Recursion continues until single-element sub-arrays are obtained.

---

**Final Sorted Output:**

25, 29, 30, 35, 42, 47, 50, 52, 60

(The array remains unchanged because it was already sorted.)

---

**Nature of the Given Input:**

- Since the input is **already sorted** and the pivot is chosen as the **first element**, the partitioning becomes **highly unbalanced**.

- Each partition results in one empty sub-array and one sub-array of size $n - 1$.
- This leads to **maximum recursion depth**.

👉 **Hence, the given input represents the *Worst Case* for Quick Sort.**

---

**Time Complexity Analysis:**

- **Best Case:** $O(n\log\ n)$
  - Occurs when pivot divides array into two equal halves.
- **Average Case:** $O(n\log\ n)$
- **Worst Case:** $O(n^2)$
  - Occurs for sorted or reverse sorted input when pivot is poorly chosen.

For the given input, the complexity is:

$$\boxed{O(n^2)}$$

---

**Advantages of Quick Sort:**

- Very fast in practice for large datasets.
- In-place sorting algorithm.
- Requires no additional memory.

---

**Disadvantages of Quick Sort:**

- Poor pivot selection leads to worst-case performance.

- Not a stable sorting algorithm.

OR

(c) What is recurrence? Explain recursion-tree method with suitable example. (7 mark)

**Recurrence and Recursion-Tree Method**

**Introduction / Definition of Recurrence:**

A recurrence is a **mathematical equation** that defines a function in terms of its values for **smaller inputs**. In algorithm analysis, a recurrence relation is used to **express the running time of a recursive algorithm** by relating the time complexity of a problem of size $n$to the time taken for solving subproblems of smaller sizes.

Recurrence relations are commonly used to analyze **divide-and-conquer algorithms** such as merge sort, quick sort, and binary search.

---

**Need of Recurrence Relations:**

- Helps in determining **time complexity** of recursive algorithms.
- Provides a clear relationship between problem size and execution time.
- Useful for comparing performance of recursive solutions.

---

**Recursion-Tree Method:**

**Definition:**
The recursion-tree method is a technique used to **solve recurrence relations** by representing the recurrence as a **tree structure**. Each node in the tree represents the **cost of a recursive call**, and the total

cost is obtained by summing the cost of all nodes at every level of the tree.

A diagram would show a root node divided into child nodes, forming a tree-like structure representing recursive calls.

---

**Steps of Recursion-Tree Method:**

1. Write the given recurrence relation.
2. Draw a recursion tree where each node represents a subproblem.
3. Label each node with the **cost of that subproblem**.
4. Calculate the cost at each level of the tree.
5. Determine the height of the tree.
6. Add the costs of all levels to obtain the total time complexity.

---

**Example using Recursion-Tree Method:**

Consider the recurrence relation:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

**Step 1: Root Level (Level 0):**

- One node with cost = $n$

**Step 2: Level 1:**

- Two subproblems each of size $\frac{n}{2}$
- Cost at each node = $\frac{n}{2}$
- Total cost at level 1 = $n$

**Step 3: Level 2:**

- Four subproblems each of size $\frac{n}{4}$
- Cost at each node = $\frac{n}{4}$
- Total cost at level 2 = $n$

**Further Levels:**

- The same pattern continues at every level.
- Each level contributes a total cost of $n$.

---

**Height of the Recursion Tree:**

- The problem size reduces by half at each level.
- Height of tree = $\log_2 n$

---

**Total Cost Calculation:**

$$T(n) = n + n + n + \cdots + n(\log\ n \text{ times})$$
$$T(n) = n\log\ n$$

---

**Final Result:**

$$\boxed{T(n) = \Theta(n\log\ n)}$$

---

**Advantages of Recursion-Tree Method:**

- Easy to visualize recursive calls.
- Helps in understanding contribution of each level.

- Useful for divide-and-conquer recurrences.

---

**Disadvantages:**

- Not suitable for complex or irregular recurrences.
- Tree drawing becomes difficult for large recursions.

---

**Applications:**

- Analysis of merge sort.
- Analysis of quick sort.
- Understanding recursive algorithm behavior.

Q.3 (a) What is Principle of Optimality? Explain its use in Dynamic Programming Method (3 mark)

**Definition / Introduction:**

The Principle of Optimality is a fundamental concept in **Dynamic Programming** which states that *an optimal solution of a problem contains optimal solutions to its subproblems*. This means that if a problem is solved optimally, then every smaller part of that solution must also be optimal.

---

**Explanation of Principle of Optimality:**

- The principle was introduced by **Richard Bellman**.
- It ensures that a complex problem can be broken down into **overlapping subproblems**.
- Each subproblem must be solved in the best possible way to achieve an optimal final result.

- If any subproblem is not optimal, the overall solution will also not be optimal.

---

**Use in Dynamic Programming Method:**

- Dynamic programming applies this principle to solve problems efficiently.
- Solutions of subproblems are **stored and reused**, avoiding repeated calculations.
- It helps in converting exponential-time recursive solutions into **polynomial-time algorithms**.
- Problems like shortest path, knapsack, and matrix chain multiplication rely on this principle.

(b) Explain Binomial Coefficient algorithm using dynamic programming. (4 mark)

**Definition / Introduction:**

The binomial coefficient represents the number of ways to choose $k$elements from a set of $n$elements without considering the order. It is denoted as $C(n,k)$or $\binom{n}{k}$. Dynamic programming is used to compute binomial coefficients efficiently by avoiding repeated calculations.

---

**Explanation of Binomial Coefficient using DP:**

- The binomial coefficient follows the recurrence relation:

$$C(n,k) = C(n-1,k-1) + C(n-1,k)$$

- The base conditions are:
  - $C(n,0) = 1$

- $C(n, n) = 1$

- This relation is derived from **Pascal's Triangle**, where each value is the sum of two values above it.

---

**Dynamic Programming Approach:**

- A two-dimensional table is constructed where rows represent values of $n$and columns represent values of $k$.
- The table is filled in a **bottom-up manner** using the recurrence relation.
- Previously computed values are stored and reused, which reduces time complexity.

---

**Algorithm (Steps in Words):**

1. Create a table $C[0..n][0..k]$.
2. Initialize $C[i][0] = 1$and $C[i][i] = 1$.
3. For remaining values, compute using:

$$C(i, j) = C(i - 1, j - 1) + C(i - 1, j)$$

4. Continue until $C(n, k)$is obtained.

---

**Advantages:**

- Avoids redundant recursive calls.
- Efficient for large values of $n$and $k$.
- Time complexity is reduced to $O(nk)$.

---

**Applications:**

- Used in probability and statistics.
- Helpful in combinatorial problems.
- Used in dynamic programming and algorithm analysis.

(c) Obtain longest common subsequence using dynamic programming. Given A = “acabaca” and B = “bacac”. (7 mark)

**Introduction / Definition:**

The **Longest Common Subsequence (LCS)** problem is a classic problem in **dynamic programming**. A subsequence is a sequence that can be derived from another sequence by deleting some or no elements **without changing the order** of the remaining elements. The LCS of two sequences is the **longest sequence that appears in both sequences in the same order**, but not necessarily consecutively.

---

**Need for Dynamic Programming:**

- The LCS problem has **overlapping subproblems** and **optimal substructure**.
- A simple recursive approach results in **exponential time complexity**.
- Dynamic programming stores intermediate results and avoids recomputation, making the solution efficient.

---

**Given Sequences:**

- Sequence A = **acabaca**
- Sequence B = **bacac**

Let the lengths be:

- Length of A = 7
- Length of B = 5

---

**LCS Recurrence Relation:**

Let $L[i][j]$represent the length of LCS of the first $i$characters of A and first $j$characters of B.

- If characters match:

$$L[i][j] = 1 + L[i-1][j-1]$$

- If characters do not match:

$$L[i][j] = \max\left(L[i-1][j], L[i][j-1]\right)$$

**Boundary Conditions:**

- $L[i][0] = 0$
- $L[0][j] = 0$

---

**Dynamic Programming Table Construction (Explained):**

- A 2D table of size $(7+1) \times (5+1)$is constructed.
- Rows correspond to characters of A and columns correspond to characters of B.
- The table is filled **row by row** using the recurrence relation.
- When characters match, diagonal values are increased by 1.
- When they do not match, the maximum of top or left cell is selected.

A diagram would show a table where matching characters form diagonal increments.

---

**DP Table Result (Final Values):**

| | | **b** | **a** | **c** | **a** | **c** |
|---|---|---|---|---|---|---|
| | 0 | 0 | 0 | 0 | 0 | 0 |
| **a** | 0 | 0 | 1 | 1 | 1 | 1 |
| **c** | 0 | 0 | 1 | 2 | 2 | 2 |
| **a** | 0 | 0 | 1 | 2 | 3 | 3 |
| **b** | 0 | 1 | 1 | 2 | 3 | 3 |
| **a** | 0 | 1 | 2 | 2 | 3 | 3 |
| **c** | 0 | 1 | 2 | 3 | 3 | 4 |
| **a** | 0 | 1 | 2 | 3 | 4 | 4 |

---

**Finding the LCS (Backtracking Explanation):**

- Start from the bottom-right cell of the table.
- If characters match, include that character in the LCS and move diagonally.
- If they do not match, move in the direction of the larger value (up or left).
- Continue until the first row or column is reached.

---

**Final Longest Common Subsequence:**

LCS = “acac”

**Length of LCS = 4**

---

**Time and Space Complexity:**

- **Time Complexity:** $O(m \times n)$
- **Space Complexity:** $O(m \times n)$
  where $m$and $n$are lengths of the two sequences.

---

**Applications of LCS:**

- File comparison tools (diff utilities).
- DNA sequence matching.
- Version control systems.
- Spell checking and text similarity analysis.

OR

Q.3 (a) Explain the steps of greedy strategy for solving a problem (3 mark)

**Definition / Introduction:**

The greedy strategy is an algorithmic technique in which a solution is constructed **step by step** by always making the **locally optimal choice** at each stage with the hope that these local choices lead to a **globally optimal solution**.

---

**Steps of Greedy Strategy:**

1. **Define the Problem and Objective Function:**

- Clearly define the problem and identify the objective to be optimized, such as minimizing cost or maximizing profit.
- Constraints of the problem are also identified at this stage.

2. **Selection of Greedy Choice:**
   - At each step, choose the option that seems **best at that moment** according to the objective function.
   - Once a choice is made, it is never reconsidered.
3. **Check Feasibility:**
   - After selecting an element, check whether it satisfies all the constraints of the problem.
   - If it violates constraints, the choice is discarded.
4. **Solution Construction:**
   - Add the selected feasible element to the solution set.
   - Repeat the greedy selection until a complete solution is obtained.
5. **Termination:**
   - The algorithm stops when the solution is complete or no more feasible choices remain.

---

(b) Demonstrate Binary Search method to search Key = 14, form the arrayA= <2,4,7,8,9,10,12,14,18>  (4 mark)

**Binary Search Method**

**Definition / Introduction:**

Binary search is an **efficient searching algorithm** used to find a specific key element in a **sorted array**. It works on the principle of **divide and conquer**, where the search space is repeatedly divided

into two halves until the required element is found or the search space becomes empty.

---

**Given Data:**

- Array $A = \langle 2,4,7,8,9,10,12,14,18 \rangle$
- Key to be searched = **14**

---

**Steps of Binary Search (Explained):**

**Step 1: Initial Setup**

- Low = 0, High = 8 (for 9 elements)
- Mid = $\lfloor (0 + 8)/2 \rfloor = 4$
- $A[4] = 9$

Since **14 > 9**, the key lies in the **right half**.

---

**Step 2: Second Iteration**

- Low = 5, High = 8
- Mid = $\lfloor (5 + 8)/2 \rfloor = 6$
- $A[6] = 12$

Since **14 > 12**, the key lies in the **right half** again.

---

**Step 3: Third Iteration**

- Low = 7, High = 8
- Mid = $\lfloor (7 + 8)/2 \rfloor = 7$
- $A[7] = 14$

The key **14 is found** at index **7** (position 8 if counted from 1).

---

**Result / Conclusion:**

The key **14** is successfully found in the array using binary search.

---

**Features of Binary Search:**

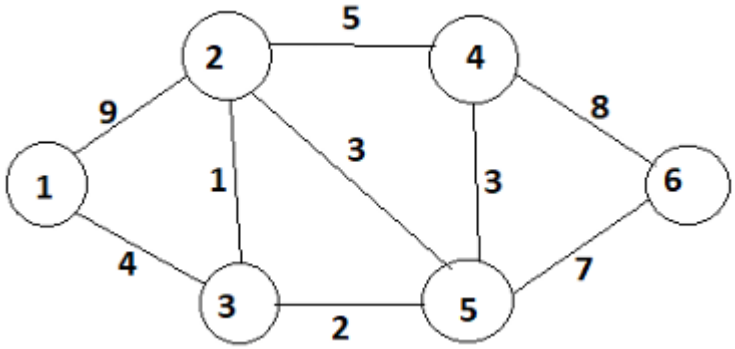
- Requires the array to be **sorted**.
- Reduces the search space by half in each step.
- Time complexity is $O(\log\ n)$.

---

**Graphical Explanation (Described):**

A diagram would show the array repeatedly divided into halves, with the search region narrowing down until the key element 14 is located.

(c) What is a minimum spanning tree? Draw the minimum spanning tree correspond to following graph using Prim's algorithm. (7 mark)




**Minimum Spanning Tree (MST) and Prim's Algorithm**

**Introduction / Definition of Minimum Spanning Tree:**

A **Minimum Spanning Tree (MST)** of a connected, undirected, weighted graph is a **spanning tree** that connects all the vertices together **without forming any cycle** and with the **minimum possible**

**total edge weight**. A spanning tree of a graph with $n$vertices always contains exactly $n - 1$edges.

The objective of an MST is to ensure **minimum cost connectivity** among all vertices.

---

**Properties of Minimum Spanning Tree:**

- Contains all vertices of the graph.
- Has exactly $n - 1$edges.
- Does not contain any cycle.
- Total weight of edges is minimum among all spanning trees.

---

**Prim's Algorithm:**

**Definition:**
Prim's algorithm is a **greedy algorithm** used to find the minimum spanning tree of a connected weighted graph. It starts with a single vertex and **grows the MST step by step** by always selecting the **minimum weight edge** that connects a vertex in the tree to a vertex outside the tree.

---

**Steps of Prim's Algorithm:**

1. Select any vertex as the starting vertex.
2. Choose the **minimum weight edge** connecting the selected vertex to an unvisited vertex.
3. Add the selected edge and vertex to the MST.
4. Repeat the process until all vertices are included in the MST.

A diagram would show the MST expanding gradually by adding minimum-weight edges.

---

**Given Graph (Vertices and Weights):**

Vertices: **1, 2, 3, 4, 5, 6**

Edges with weights (from the graph):

- 1–2 : 9
- 1–3 : 4
- 2–3 : 1
- 2–4 : 5
- 2–5 : 3
- 3–5 : 2
- 4–5 : 3
- 4–6 : 8
- 5–6 : 7

---

**Construction of MST using Prim's Algorithm (Step-by-Step):**

Assume **Vertex 1** as the starting point.

**Step 1:**

- Start with vertex **1**
- Available edges: (1–2 = 9), (1–3 = 4)
- Select minimum → **1–3 (4)**

MST edges: {1–3}

---

**Step 2:**

- Vertices included: {1, 3}
- Possible edges:
  - 3–2 = 1
  - 3–5 = 2
- Select minimum → **3–2 (1)**

MST edges: {1–3, 3–2}

---

**Step 3:**

- Vertices included: {1, 2, 3}
- Possible edges:
  - 3–5 = 2
  - 2–5 = 3
  - 2–4 = 5
- Select minimum → **3–5 (2)**

MST edges: {1–3, 3–2, 3–5}

---

**Step 4:**

- Vertices included: {1, 2, 3, 5}
- Possible edges:
  - 5–4 = 3
  - 5–6 = 7
- Select minimum → **5–4 (3)**

MST edges: {1–3, 3–2, 3–5, 5–4}

---

**Step 5:**

- Vertices included: {1, 2, 3, 4, 5}
- Possible edges:
  - 5–6 = 7
  - 4–6 = 8
- Select minimum → **5–6 (7)**

MST edges: {1–3, 3–2, 3–5, 5–4, 5–6}

---

**Final Minimum Spanning Tree:**

**Edges in MST:**

- 1–3 (4)
- 3–2 (1)
- 3–5 (2)
- 5–4 (3)
- 5–6 (7)

**Total Cost of MST:**

$$4 + 1 + 2 + 3 + 7 = \boxed{17}$$

A diagram would show these edges highlighted, connecting all vertices without cycles.

---

**Advantages of Prim's Algorithm:**

- Simple and easy to understand.

- Efficient for dense graphs.
- Guarantees minimum cost connectivity.

---

**Applications of MST:**

- Network design (computer networks, telephone lines).
- Road and railway planning.
- Electrical circuit design.
- Cluster analysis.

---

Q.4 (a) Explain Tower of Hanoi Problem, Derive its recursion equation and computer it's time complexity. (3 mark)

**Tower of Hanoi Problem**

**Introduction / Definition:**

The **Tower of Hanoi** is a classic problem in computer science that demonstrates the concept of **recursion**. It consists of three rods and a number of disks of different sizes, which can slide onto any rod. The objective is to move all the disks from the **source rod** to the **destination rod** using the **auxiliary rod**, following a specific set of rules.

---

**Rules of the Problem:**

- Only **one disk** can be moved at a time.
- A larger disk **cannot be placed on top of a smaller disk**.
- All disks must be moved from the source rod to the destination rod.

---

**Recursion Equation (Derivation):**

Let $T(n)$be the number of moves required to transfer $n$disks.

- Move top $n-1$disks from source to auxiliary rod $\rightarrow T(n-1)$
- Move the largest disk from source to destination $\rightarrow$ 1 move
- Move $n-1$disks from auxiliary to destination $\rightarrow T(n-1)$

Thus, the recurrence relation is:

$$T(n) = 2T(n-1) + 1$$

---

**Time Complexity:**

- Solving the recurrence gives:

$$T(n) = 2^n - 1$$

- Therefore, the **time complexity** of the Tower of Hanoi problem is:
- $\boxed{O(2^n)}$
-

(b) Using greedy algorithm find an optimal schedule for following jobs with n=7 profits: (P1, P2, P3, P4, P5, P6, P7) = (3, 5, 18, 20, 6, 1, 38) and deadline (d1, d2, d3, d4, d5, d6, d7) = (1, 3, 3, 4, 1, 2, 1) (4 mark)

**Job Sequencing with Deadlines using Greedy Algorithm**

**Introduction / Definition:**

Job sequencing with deadlines is a **greedy optimization problem** where a set of jobs must be scheduled in such a way that **maximum total profit** is obtained. Each job takes **one unit of time**, and every

job has a **deadline** before which it must be completed to earn its profit.

The greedy approach selects jobs based on **maximum profit first** and schedules them within their deadlines.

---

**Given Data:**

Number of jobs: $n = 7$

| Job | Profit | Deadline |
|---|---|---|
| P1 | 3 | 1 |
| P2 | 5 | 3 |
| P3 | 18 | 3 |
| P4 | 20 | 4 |
| P5 | 6 | 1 |
| P6 | 1 | 2 |
| P7 | 38 | 1 |

---

**Step 1: Sort Jobs in Decreasing Order of Profit**

| Job | Profit | Deadline |
|---|---|---|
| P7 | 38 | 1 |
| P4 | 20 | 4 |
| P3 | 18 | 3 |
| P5 | 6 | 1 |

| Job | Profit | Deadline |
|---|---|---|
| P2 | 5 | 3 |
| P1 | 3 | 1 |
| P6 | 1 | 2 |

---

**Step 2: Find Maximum Deadline**

Maximum deadline = **4**
So, available time slots = **4** (Slot 1, 2, 3, 4)

---

**Step 3: Job Allocation (Greedy Scheduling)**

- **P7 (38, d=1)** → Slot 1
- **P4 (20, d=4)** → Slot 4
- **P3 (18, d=3)** → Slot 3
- **P5 (6, d=1)** → Slot 1 (occupied ❌, cannot be scheduled)
- **P2 (5, d=3)** → Slot 2 (free ✔)
- **P1 (3, d=1)** → Slot 1 (occupied ❌)
- **P6 (1, d=2)** → Slot 2 (occupied ❌)

---

**Final Job Schedule:**

| Time Slot | Job |
|---|---|
| 1 | P7 |
| 2 | P2 |

| Time Slot | Job |
|---|---|
| 3 | P3 |
| 4 | P4 |

---

**Total Profit Calculation:**

$$38 + 5 + 18 + 20 = \boxed{81}$$

---

(c) Solve the following Knapsack Problem using greedy method. Number of items = 5, knapsack capacity W = 100, weight vector = {50, 40, 30, 20, 10} and profit vector = {1, 2, 3, 4, 5}. (7 mark)

**Knapsack Problem Using Greedy Method**

**Introduction / Definition**

The Knapsack Problem is a classic optimization problem in computer science and operations research. The objective is to maximize the total profit of items that can be placed in a knapsack with a limited capacity. There are two main types: **0/1 Knapsack** (where items cannot be divided) and **Fractional Knapsack** (where items can be divided). The **Greedy Method** is particularly suited for the Fractional Knapsack problem, as it selects items based on the best profit-to-weight ratio until the knapsack is full.

The **problem given** is:

- Number of items, n = 5
- Knapsack capacity, W = 100
- Weight vector: {50, 40, 30, 20, 10}

- Profit vector: {1, 2, 3, 4, 5}

Our goal is to find the maximum profit using a greedy approach.

---

**Step 1: Calculate Profit-to-Weight Ratio**

The Greedy Method selects items based on the **profit/weight (p/w) ratio**. This ratio helps us choose items that provide the highest profit per unit weight.

| Item | Weight (w) | Profit (p) | Profit/Weight (p/w) |
|---|---|---|---|
| 1 | 50 | 1 | 0.02 |
| 2 | 40 | 2 | 0.05 |
| 3 | 30 | 3 | 0.10 |
| 4 | 20 | 4 | 0.20 |
| 5 | 10 | 5 | 0.50 |

---

**Step 2: Sort Items by Profit-to-Weight Ratio**

Items are sorted in **descending order** of p/w ratio:

**Item 5 → Item 4 → Item 3 → Item 2 → Item 1**

---

**Step 3: Select Items Using Greedy Approach**

1. Start with **knapsack capacity W = 100**.
2. Pick **Item 5** (weight = 10, profit = 5) → remaining capacity = 100 − 10 = 90.
3. Pick **Item 4** (weight = 20, profit = 4) → remaining capacity = 90 − 20 = 70.

4. Pick **Item 3** (weight = 30, profit = 3) → remaining capacity = 70 − 30 = 40.
5. Pick **Item 2** (weight = 40, profit = 2) → remaining capacity = 40 − 40 = 0.

At this point, the knapsack is full. **Item 1 cannot be included** as there is no remaining capacity.

---

**Step 4: Calculate Total Profit**

- Total profit = 5 + 4 + 3 + 2 = **14**
- Total weight = 10 + 20 + 30 + 40 = 100 (full capacity utilized)

Hence, the **maximum profit** using the greedy method is **14**.

---

**Key Features of Greedy Method**

- Always chooses the **locally optimal solution** hoping for a global optimum.
- Works well for **Fractional Knapsack** problems.
- May not guarantee optimal solution for **0/1 Knapsack**.

**Advantages:**

- Simple and easy to implement.
- Fast and efficient for problems like fractional knapsack.

**Disadvantages:**

- Not always optimal for all knapsack variations.
- Only suitable when **greedy choice property** and **optimal substructure** hold.

---

**Applications**

- Resource allocation problems.
- Investment decisions with budget constraints.
- Cargo loading optimization.
- Network bandwidth allocation.

---

**Example Explanation**

If we visualize a diagram:

- Imagine a rectangle representing the knapsack of capacity 100.
- Items are filled starting with the highest p/w ratio (Item 5) first, then Item 4, and so on.
- Each item occupies a portion of the rectangle proportional to its weight.
- After placing Items 5, 4, 3, and 2, the rectangle is completely filled, showing maximum profit is achieved.

---

**Answer:**

- Items selected: **5, 4, 3, 2**
- Total weight = **100**
- Total profit = **14**

Q.4 (a) What are the disadvantages of greedy method over dynamic programming method? (3 mark)

**Introduction**

The Greedy Method is a popular optimization technique where the **best choice is made at each step**, hoping to achieve a global

optimum. However, unlike the **Dynamic Programming (DP) method**, which considers **all possible subproblems** and guarantees an optimal solution, the greedy method has several limitations. Understanding these disadvantages is important when choosing an appropriate problem-solving technique.

---

**Disadvantages**

1. **May Not Give Global Optimum**
    - Greedy method only makes **locally optimal choices**.
    - It does not always produce the overall optimal solution.
    - Example: In the 0/1 Knapsack Problem, greedy may fail to select the best combination of items.
2. **Problem-Specific Applicability**
    - Greedy works only when a problem satisfies **greedy-choice property** and **optimal substructure**.
    - Many problems require checking all combinations, where greedy fails but DP succeeds.
3. **No Backtracking**
    - Once a choice is made, the greedy method **cannot reconsider it**.
    - In contrast, dynamic programming examines all subproblems and can correct previous decisions.
4. **Suboptimal for Complex Problems**
    - For problems with dependencies or constraints between choices, greedy may ignore better alternatives.
    - Dynamic programming ensures **all constraints** and **dependencies** are handled.

(b) Find an optimal Huffman code for the following set of frequency. A : 50, b: 20, c: 15, d: 30 (4 mark)

**Optimal Huffman Code**

**Introduction / Definition**

Huffman coding is a **lossless data compression technique** used to reduce the average length of codewords based on the frequency of characters. The characters with **higher frequency** are assigned **shorter codes**, and characters with lower frequency are assigned **longer codes**. This ensures an **efficient binary encoding** that minimizes the total number of bits required.

We are given the following frequencies:

- A: 50
- B: 20
- C: 15
- D: 30

Our goal is to construct an **optimal Huffman code**.

---

**Step 1: Arrange Frequencies in Ascending Order**

| Character | Frequency |
|---|---|
| C | 15 |
| B | 20 |
| D | 30 |
| A | 50 |

---

**Step 2: Build Huffman Tree**

1. **Pick two lowest frequencies**: C (15) and B (20) → combine to form a new node with frequency = 15 + 20 = 35.
2. Remaining nodes: 35, D (30), A (50) → pick lowest two: D (30) and 35 → combine → new node frequency = 30 + 35 = 65.
3. Remaining nodes: 65 and A (50) → combine → root node frequency = 65 + 50 = 115.

---

**Step 3: Assign Binary Codes**

- Start from the root:
    - Assign **0** to the left branch, **1** to the right branch at each node.
- Trace the path from root to each leaf:

| Character | Code |
|---|---|
| A | 1 |
| D | 01 |
| B | 001 |
| C | 000 |

**Explanation:**

- A has the **highest frequency**, so it gets the **shortest code** (1).
- C has the **lowest frequency**, so it gets the **longest code** (000).
- Codes are **prefix-free**, ensuring no ambiguity during decoding.

---

**Key Features of Huffman Coding**

- **Prefix Property**: No code is a prefix of another code.
- **Optimality**: Produces **minimum average code length**.
- **Binary Tree Structure**: Can be represented as a binary tree with frequencies as node values.

**Advantages:**

- Reduces storage space and transmission time.
- Simple and efficient for known frequencies.

**Disadvantages:**

- Requires **frequency table** beforehand.
- Less effective for **small datasets**.

---

**Example Explanation**

Imagine a tree diagram:

- Root node = 115
- Left child = 65 → left = 30 (D), right = 35 → left = 15 (C), right = 20 (B)
- Right child of root = 50 (A)

This diagram visually shows the **binary paths** that lead to each character, matching the codes above.

---

**Answer:**

| Character | Huffman Code |
|---|---|
| A | 1 |
| B | 001 |

| Character | Huffman Code |
|---|---|
| C | 000 |
| D | 01 |

(c) Consider a Knapsack with the maximum weight capacity M is 7, for the objects with value [12,10,20,15] with weights [2,1,3,2] solve using dynamic programming the maximum value the knapsack can have. (7 mark)

**0/1 Knapsack Problem Using Dynamic Programming**

**Introduction / Definition**

The **0/1 Knapsack Problem** is a classic optimization problem where a knapsack has a **maximum weight capacity** and a set of items, each with a **weight** and a **value**. The goal is to **maximize the total value** without exceeding the weight capacity.

Unlike the greedy method, the **Dynamic Programming (DP) approach** guarantees an **optimal solution** by considering all combinations of items using a table-based method. DP uses the principle of **optimal substructure**, storing solutions of subproblems to avoid recomputation.

We are given:

- Maximum weight capacity, $M = 7$
- Values: $[12, 10, 20, 15]$
- Weights: $[2, 1, 3, 2]$

---

**Step 1: Create DP Table**

Let $V[i][w]$represent the **maximum value** achievable with the first $i$items and capacity $w$.

- Number of items $n = 4$
- Table dimensions: $(n + 1) \times (M + 1) = 5 \times 8$

**Initialization:**

- $V[0][w] = 0$for all w (0 items → 0 value)
- $V[i][0] = 0$for all i (0 capacity → 0 value)

---

**Step 2: Fill DP Table Using Recurrence**

**Recurrence relation:**

$$V[i][w] = \begin{cases} V[i-1][w], & \text{if weight[i] > w} \\ \max\left(V[i-1][w], V[i-1][w - weight[i]] + value[i]\right), & \text{if weight[i] } \leq \text{ w} \end{cases}$$

---

**Step 3: Table Computation**

**Items and weights:**

| Item | Weight | Value |
|---|---|---|
| 1 | 2 | 12 |
| 2 | 1 | 10 |
| 3 | 3 | 20 |
| 4 | 2 | 15 |

Compute $V[i][w]$step by step:

**Item 1 (weight 2, value 12):**

- w = 1 → cannot include → $V[1][1] = 0$
- w = 2 → include → $V[1][2] = 12$

- w = 3 → include → $V[1][3] = 12$
- w = 4 → include → $V[1][4] = 12$
- w = 5 → include → $V[1][5] = 12$
- w = 6 → include → $V[1][6] = 12$
- w = 7 → include → $V[1][7] = 12$

**Item 2 (weight 1, value 10):**

- w = 1 → include → $V[2][1] = 10$
- w = 2 → max(12, 10) = 12
- w = 3 → max(12, 10+0) = 12
- w = 4 → max(12, 10+12) = 22
- w = 5 → max(12, 10+12) = 22
- w = 6 → max(12, 10+12) = 22
- w = 7 → max(12, 10+12) = 22

**Item 3 (weight 3, value 20):**

- w = 1,2 → cannot include → copy previous row
- w = 3 → include → max(22, 20+0) = 20 → take max with previous row → 20
- w = 4 → max(22, 20+10)=30
- w = 5 → max(22, 20+12)=32
- w = 6 → max(22, 20+12)=32
- w = 7 → max(22, 20+22)=42

**Item 4 (weight 2, value 15):**

- w = 1 → cannot include → 0
- w = 2 → include → max(12, 15)=15

- w = 3 → max(12, 15+0)=15
- w = 4 → max(22, 15+10)=25
- w = 5 → max(22, 15+12)=27
- w = 6 → max(32, 15+22)=37
- w = 7 → max(42, 15+30)=45

---

**Step 4: Maximum Value**

The **maximum value** the knapsack can hold is:

$$V[4][7] = 45$$

---

**Step 5: Selected Items (Optional Backtracking)**

- Start from $V[4][7] = 45$
- Check which items contributed:
  - Item 4 included → weight 2 → remaining capacity = 5 → value left = 30
  - Item 3 included → weight 3 → remaining capacity = 2 → value left = 10
  - Item 2 included → weight 1 → remaining capacity = 1 → value left = 0
- Selected items: **Item 2, Item 3, Item 4**

---

**Key Features of Dynamic Programming Method**

- Considers **all subproblems**, guarantees **optimal solution**

- Uses a **table to store intermediate results**, avoiding recomputation
- Works for **0/1 Knapsack**, unlike greedy

**Advantages:**

- Always gives **maximum possible value**
- Efficient for **small to medium size problems**

**Disadvantages:**

- Requires **extra memory** for table
- Time complexity increases with large n and W

---

**Applications**

- Resource allocation problems
- Investment and budget optimization
- Cargo loading and logistics
- Memory allocation in computing

---

**Answer:**

- **Maximum Value** = **45**
- **Selected Items** = **2, 3, 4**
- **Knapsack Capacity Used** = 7

Q.5 (a) Define NP-Complete and NP-Hard problems. (3 mark)

**NP-Complete and NP-Hard Problems**

**Introduction / Definition**

In computational complexity theory, problems are classified based on how difficult they are to solve and verify. Two important classes are **NP-Complete** and **NP-Hard** problems. Understanding these helps in designing algorithms and knowing the limitations of computation.

---

**1. NP-Complete Problems**

- **Definition:** NP-Complete (Non-deterministic Polynomial-time Complete) problems are those that are:
    1. In **NP** (Non-deterministic Polynomial-time), meaning a proposed solution can be **verified in polynomial time**.
    2. **As hard as any problem in NP**, i.e., any NP problem can be **reduced to it in polynomial time**.
- **Key Point:** If an NP-Complete problem can be solved in polynomial time, **all NP problems can also be solved in polynomial time**.
- **Example:** Traveling Salesman Problem, 0/1 Knapsack Problem, Boolean Satisfiability (SAT).

---

**2. NP-Hard Problems**

- **Definition:** NP-Hard problems are at least **as hard as NP-Complete problems**, but they **may or may not be in NP**.
- **Key Point:** NP-Hard problems **do not require a solution to be verifiable in polynomial time**.
- **Example:** Halting Problem, Optimization version of Traveling Salesman Problem.

---

**Comparison / Important Notes**

| Feature | NP-Complete | NP-Hard |
|---|---|---|
| Solution verification | Polynomial time | Not necessarily polynomial |
| Part of NP class | Yes | Not necessarily |
| Hardness | Hardest problems in NP | At least as hard as NP-Complete |
| Examples | SAT, 0/1 Knapsack | Halting Problem, TSP (optimization) |

(b) Draw the state space tree Diagram for 4 Queens problem. (4 mark)

**tate Space Tree for 4-Queens Problem**

**Introduction / Definition**

The **N-Queens Problem** is a classic combinatorial problem where **N queens** must be placed on an **N×N chessboard** such that **no two queens attack each other**. A queen can attack along the **same row, column, or diagonal**.

The **state space tree** is a **tree representation of all possible placements** of queens, used in **backtracking algorithms** to systematically explore solutions. Each node in the tree represents a **partial placement of queens**, and branches represent **choices for the next queen**.

---

**Explanation of State Space Tree for 4-Queens**

1. **Level 1 (Row 1):**
    - Place the first queen (Q1) in **column 1, 2, 3, or 4**. Each placement creates a separate branch from the root.
2. **Level 2 (Row 2):**

    - For each Q1 placement, place the second queen (Q2) in a **column not attacked by Q1**.
    - Only valid positions are considered. Invalid positions are **pruned**.
3. **Level 3 (Row 3):**
    - For each partial solution of 2 queens, place the third queen (Q3) in a safe column.
    - Again, **prune branches** where Q3 is under attack.
4. **Level 4 (Row 4):**
    - Place the fourth queen (Q4) in a safe column for each 3-queen configuration.
    - Only the **complete solutions** reach the leaf nodes of the tree.

---

**Description of the Tree Structure**

- **Root Node:** Empty board (0 queens placed)
- **Branches:** Represent column choices for placing a queen in the next row
- **Pruned Branches:** Any branch where a queen would be attacked is **not expanded**
- **Leaf Nodes:** Represent **valid solutions** where all 4 queens are safely placed

**Example of Solutions (represented in columns per row):**

1. Q1 → Column 2, Q2 → Column 4, Q3 → Column 1, Q4 → Column 3

2. Q1 → Column 3, Q2 → Column 1, Q3 → Column 4, Q4 → Column 2

---

**Key Features of State Space Tree**

- Helps **visualize the backtracking process**
- Reduces computation by **pruning invalid branches early**
- Shows all **partial and complete solutions**

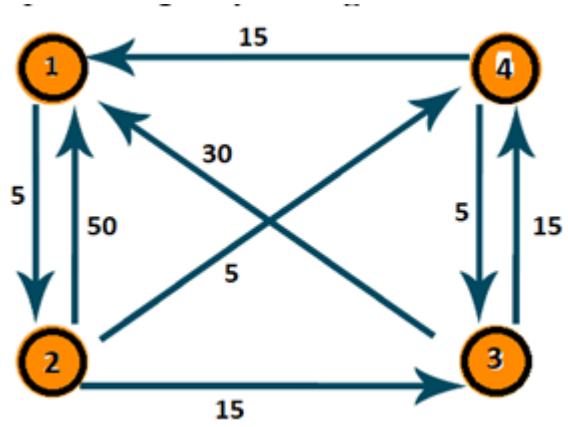
**Advantages:**

- Systematic approach to solve N-Queens
- Easy to implement using recursion and backtracking

**Disadvantages:**

- Tree size grows **exponentially** with N
- Requires more memory for large N

(c) Find all pair of shortest path using Floyd's Algorithm for following graph. (7 mark)




**Introduction / Definition**

Floyd's Algorithm, also known as the **Floyd-Warshall algorithm**, is a **dynamic programming technique** used to find the **shortest paths between all pairs of vertices** in a weighted graph.

- It works for **both directed and undirected graphs**.
- Can handle **positive and negative edge weights** but **no negative cycles**.
- The algorithm uses a **distance matrix** that is updated iteratively considering each vertex as an intermediate node.

---

**Step 1: Represent Graph as Distance Matrix**

Let the vertices be 1,2,3,4.

- Use ∞for no direct edge.

**Initial Distance Matrix (D0):**

| | **1** | **2** | **3** | **4** |
|---|---|---|---|---|
| 1 | 0 | 5 | ∞ | 15 |
| 2 | 5 | 0 | 15 | ∞ |
| 3 | ∞ | ∞ | 0 | 5 |
| 4 | 3 | ∞ | 5 | 0 |

---

**Step 2: Apply Floyd's Algorithm**

**Recurrence formula:**

$$D[i][j] = \min\left(D[i][j], D[i][k] + D[k][j]\right)$$

- For each intermediate vertex $k = 1$to $4$

- Update all pairs $(i, j)$

**Iteration k = 1**

- Check if using vertex 1 as intermediate reduces any distance.
- Updates:
  - D[2][1] = min(50, D[2][1]+D[1][1]) = min(50, 50+0)=50 (no change)
  - D[2][4] = min(∞, D[2][1]+D[1][4]) = min(∞, 50+15)=65

**Updated Matrix after k=1:**

| | **1** | **2** | **3** | **4** |
|---|---|---|---|---|
| 1 | 0 | 5 | ∞ | 15 |
| 2 | 50 | 0 | 15 | 65 |
| 3 | ∞ | ∞ | 0 | 5 |
| 4 | 30 | ∞ | 5 | 0 |

---

**Iteration k = 2**

- Use vertex 2 as intermediate:
- Updates:
  - D[1][3] = min(∞, D[1][2]+D[2][3]) = min(∞, 5+15)=20
  - D[4][2] = min(∞, D[4][1]+D[1][2]) = min(∞, 30+5)=35

**Matrix after k=2:**

| | **1** | **2** | **3** | **4** |
|---|---|---|---|---|
| 1 | 0 | 5 | 20 | 15 |
| 2 | 50 | 0 | 15 | 65 |

| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 3 | ∞ | ∞ | 0 | 5 |
| 4 | 30 | 35 | 5 | 0 |

---

**Iteration k = 3**

- Use vertex 3 as intermediate:
- Updates:
  - D[1][4] = min(15, D[1][3]+D[3][4]) = min(15, 20+5)=15 (no change)
  - D[2][4] = min(65, 15+5)=20
  - D[4][1] = min(30, 5+50)=30 (no change)
- Other distances remain same

**Matrix after k=3:**

| | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | 0 | 5 | 20 | 15 |
| 2 | 50 | 0 | 15 | 20 |
| 3 | ∞ | ∞ | 0 | 5 |
| 4 | 30 | 35 | 5 | 0 |

---

**Iteration k = 4**

- Use vertex 4 as intermediate:
- Updates:

- D[2][1] = min(50, D[2][4]+D[4][1]) = min(50, 20+30)=50 (no change)
- D[3][1] = min(∞, 5+30)=35
- D[3][2] = min(∞, 5+35)=40

- Other distances remain same

**Final Distance Matrix (All-Pairs Shortest Path):**

| | **1** | **2** | **3** | **4** |
|---|---|---|---|---|
| 1 | 0 | 5 | 20 | 15 |
| 2 | 50 | 0 | 15 | 20 |
| 3 | 35 | 40 | 0 | 5 |
| 4 | 30 | 35 | 5 | 0 |

---

**Step 3: Interpretation**

- Each entry D[i][j] gives the **shortest distance** from vertex i to vertex j.
- For example:
  - Shortest path from 1 → 3 = 20
  - Shortest path from 3 → 2 = 40

---

**Key Features of Floyd's Algorithm**

- Finds **shortest paths for all pairs** in a single execution
- Works for **directed and weighted graphs**
- Uses **dynamic programming**, updates distance matrix iteratively

- Handles **positive and negative edge weights** (no negative cycles)

**Advantages:**

- Simple and systematic
- Works for **dense graphs** efficiently

**Disadvantages:**

- Requires **$O(n^3)$ time complexity**
- Uses **$O(n^2)$ space**

OR

Q.5 (a) Define BFS. How it is differ from DFS. (3 mark)

**Introduction / Definition**

**Breadth-First Search (BFS)** is a **graph traversal algorithm** used to explore all vertices and edges of a graph systematically.

- BFS starts from a **source vertex** and explores all **neighboring vertices at the current level** before moving to the next level.
- It uses a **queue** data structure to keep track of vertices to be visited.
- BFS is widely used to find **shortest paths in unweighted graphs** and for **level-order traversal** in trees.

**Algorithm Steps:**

1. Start at the source vertex and mark it as visited.
2. Enqueue the source vertex in a queue.
3. While the queue is not empty:
    - Dequeue a vertex and process it.

- Enqueue all its unvisited adjacent vertices and mark them as visited.

---

**Difference Between BFS and DFS**

| Feature | BFS (Breadth-First Search) | DFS (Depth-First Search) |
|---|---|---|
| Data Structure | Queue | Stack (or recursion) |
| Traversal Method | Level by level (horizontal) | Depth-wise (vertical) |
| Path Finding | Finds **shortest path** in unweighted graphs | May not find shortest path |
| Memory Usage | More memory for storing all neighbors | Less memory in sparse graphs |
| Use Cases | Shortest path, network broadcasting | Topological sort, cycle detection |

(b) Explain the naive string matching algorithm. (4 mark)

**Naive String Matching Algorithm**

**Definition:**
The Naive String Matching Algorithm is one of the simplest methods used to find all occurrences of a pattern P of length m in a text T of length n. The algorithm works by checking for a match of the pattern at every possible position in the text, one by one, without using any preprocessing or advanced techniques.

**Explanation:**

- The algorithm starts with the first character of the text and aligns the pattern with it.

- It compares the characters of the pattern with the corresponding characters in the text sequentially.
- If all characters match, the position is recorded as a match.
- If a mismatch occurs, the pattern shifts by one position to the right in the text, and comparison starts again from the first character of the pattern.
- This process continues until the end of the text is reached, i.e., until n - m + 1 positions are checked.

**Steps / Procedure:**

1. Start with the first position in the text.
2. Compare the first character of the pattern with the current character of the text.
3. Continue comparison for all characters of the pattern.
4. If all characters match, report the starting position as a match.
5. If mismatch occurs, shift the pattern by one position to the right.
6. Repeat steps 2–5 until the end of the text.

**Key Points / Features:**

- Simple and easy to implement.
- Does not require any preprocessing of the pattern or text.
- Works for small texts or patterns efficiently.

**Advantages:**

- Straightforward and easy to understand.
- No extra memory is required apart from storing text and pattern.

**Disadvantages:**

- Inefficient for large texts and patterns.

- Time complexity is **O((n - m + 1) × m)** in the worst case.
- Can be slow if the text contains repeated characters or patterns.

**Example (Short):**

- Text T = "ABABAB" and Pattern P = "AB"
- The algorithm will check positions 1, 2, 3, 4, 5 in the text.
- Matches are found at positions 1, 3, and 5.

(c) Explain spurious hits in Rabin-Karp string matching algorithm with example. Working modulo q=13, how many spurious hits does the Rabin Karp matcher encounter in the text T = 23590231415267399921 when looking for the pattern P = 31415? (7 mark)

**Spurious Hits in Rabin-Karp String Matching Algorithm**

**Introduction / Definition:**
The **Rabin-Karp String Matching Algorithm** is a popular algorithm for finding a pattern P of length m in a text T of length n. It uses a **hash function** to convert the pattern and text substrings into numeric values and then compares these hash values to quickly detect matches.

A **spurious hit** occurs when the hash value of a substring in the text matches the hash value of the pattern, but the actual substring does **not** match the pattern. In other words, it is a **false positive** generated due to hash collisions.

---

**Detailed Explanation**

- **Rabin-Karp Process:**
    1. Compute the hash value of the pattern P.
    2. Compute the hash value of each substring of length m in text T.

3. Compare the hash of the pattern with the hash of each substring:
    - If the hash values match, check the actual characters to confirm.
    - If the characters do not match, it is a **spurious hit**.
4. Slide the pattern one position to the right and repeat.

- **Importance of spurious hits:**
    - They slow down the algorithm because after a hash match, a **character-by-character verification** is required.
    - Choosing a good modulus q reduces the probability of spurious hits.

---

**Key Features of Rabin-Karp Algorithm**

- Uses **hashing** for efficient pattern detection.
- Time complexity:
    - Best case: O(n + m)
    - Worst case: O(n × m) (when many spurious hits occur)
- Efficient for **multiple pattern matching**.

---

**Example Problem**

**Given:**

- Text: T = 23590231415267399 21
- Pattern: P = 31415
- Modulus: q = 13

**Step 1: Compute Hash of Pattern**

- Convert the pattern into numeric representation (assuming direct digits):
- P = 31415
- Hash(P) mod 13 = 31415 mod 13
- Divide 31415 by 13:
- 13 × 2416 = 31408
- 31415 - 31408 = 7
- **Hash(P) = 7**

**Step 2: Compute Hash of Each Substring in T**

- Substrings of length 5 in T (5 consecutive digits each):
    1. 23590 → 23590 mod 13 = 5
    2. 35902 → 35902 mod 13 = 6
    3. 59023 → 59023 mod 13 = 1
    4. 90231 → 90231 mod 13 = 7 → Hash matches P → **spurious hit** (Check characters: 90231 ≠ 31415)
    5. 02314 → 2314 mod 13 = 12
    6. 23141 → 23141 mod 13 = 4
    7. 31415 → 31415 mod 13 = 7 → Hash matches and substring matches → **actual hit**
    8. Remaining substrings are checked similarly.

**Step 3: Count Spurious Hits**

- From the above, **spurious hits** occur at positions where hash matches but characters don't:
    - Substring 90231 → spurious hit
- **Total spurious hits = 1**

---

**Features / Advantages of Rabin-Karp**

- Efficient for multiple pattern search.
- Simple to implement using modular arithmetic.
- Reduces unnecessary comparisons via hash values.

**Disadvantages:**

- Spurious hits may increase comparisons.
- Efficiency depends on the choice of modulus q.

---

**Applications**

- Text search in documents or DNA sequences.
- Plagiarism detection in code or text.
- Pattern matching in database strings.

---

**Small Example in Words**

- Suppose we are searching for 31415 in 2359023141526739921.
- The algorithm computes hash for 31415 → 7.
- Each substring's hash is compared.
- At substring 90231, hash = 7 but actual text ≠ pattern → spurious hit.
- At substring 31415, hash = 7 and text = pattern → match found.